Projet MLSF : résolution du jeu Meowdoku

Question 1 : modélisation, conversion et génération de grilles

1 Objectif et choix de conception

L'objectif de cette question est de traduire les règles du Meowdoku en une formule propositionnelle en forme normale conjonctive (CNF), puis de produire un fichier DIMACS utilisable par un solveur SAT. Il faut également construire des grilles dont l'existence d'une solution est garantie.

Nous distinguons deux programmes. Le fichier `grid_convertion.py` lit une grille et produit son encodage DIMACS. Cette conversion accepte le cas général d'une grille de N lignes, M colonnes et K régions. Le fichier `grid_generation.py` génère des grilles. Pour cette seconde tâche, nous retenons la simplification autorisée par l'énoncé : $N = M = K = n$. Le programme fonctionne avec Python 3.12 et utilise seulement la bibliothèque standard.

2 Modélisation propositionnelle

2.1 Variables et signification

Pour chaque case (i, j), avec $1 \leq i \leq N$ et $1 \leq j \leq M$, nous introduisons une variable booléenne $x_{i,j}$. Elle vaut vrai si la case contient un chat et faux sinon. Le nombre de variables est donc NM. Les régions $R_1, \ldots, R_K$ constituent une partition des cases : chaque case appartient à une seule région et chaque région est non vide.

La formule finale est la conjonction de toutes les contraintes présentées ci-dessous. Chaque contrainte est déjà écrite sous forme de clause ou de conjonction de clauses ; aucune transformation supplémentaire en CNF n'est nécessaire.

2.2 Au plus un chat par ligne

Pour une ligne donnée, nous interdisons que deux cases distinctes contiennent simultanément un chat. Pour chaque i et chaque paire $j < \ell$, nous ajoutons :

$$\neg x_{i,j} \vee \neg x_{i,\ell}.$$

Cette clause est fausse seulement lorsque les deux variables valent vrai. Elle autorise donc zéro ou un chat parmi les deux cases. En considérant toutes les paires de la ligne, elle garantit au plus un chat dans cette ligne.

Par exemple, pour trois cases de variables a, b, c, les clauses sont $(\neg a \vee \neg b)$, $(\neg a \vee \neg c)$ et $(\neg b \vee \neg c)$. Aucune clause positive n'est ajoutée pour imposer un chat par ligne dans le cas général : l'énoncé demande seulement « au plus un ».

2.3 Au plus un chat par colonne

Le même principe est appliqué aux colonnes. Pour chaque colonne j et chaque paire de lignes $i < k$, nous ajoutons :

$$\neg x_{i,j} \vee \neg x_{k,j}.$$

Les contraintes de lignes et de colonnes sont indépendantes des noms et des formes des régions.

2.4 Exactement un chat par région

Pour chaque région R_r, nous combinons deux exigences. La première impose *au moins un chat* :

$$\bigvee_{(i,j)\in R_r} x_{i,j}.$$

La seconde impose *au plus un chat*. Pour chaque paire de cases distinctes u, v de cette région, nous ajoutons :

$$\neg x_u \vee \neg x_v.$$

Ainsi, une région contenant les variables a, b, c fournit la clause $(a \vee b \vee c)$ et les trois clauses d'interdiction par paires. La clause positive exclut une région vide de chats ; les clauses négatives excluent une région avec plusieurs chats. Ensemble, elles imposent exactement un chat. Pour une région réduite à une seule case, la clause positive est unitaire et impose directement un chat sur cette case.

2.5 Absence de chats adjacents

Deux cases distinctes (i, j) et (k, ℓ) sont voisines au sens du 8-voisinage lorsque

$$\max(|i - k|, |j - \ell|) = 1.$$

Elles ne doivent pas contenir deux chats simultanément. Les voisinages horizontaux sont déjà couverts par les contraintes de lignes et les voisinages verticaux par les contraintes de colonnes. Il suffit donc d'ajouter les deux types de voisinages diagonaux :

$$\neg x_{i,j} \vee \neg x_{i+1,j+1}, \qquad \neg x_{i,j+1} \vee \neg x_{i+1,j},$$

pour $1 \leq i < N$ et $1 \leq j < M$. Considérer les diagonales vers la ligne suivante suffit : chaque paire diagonale est alors traitée une seule fois.

2.6 Correction de l'encodage

Si une valuation satisfait la formule, nous plaçons un chat sur chaque case dont la variable vaut vrai. Les clauses par paires garantissent au plus un chat par ligne et par colonne ainsi que l'absence de voisinage interdit. Les clauses des régions garantissent exactement un chat dans chacune d'elles. Le placement respecte donc toutes les règles du jeu.

Réciproquement, un placement valide définit une valuation en mettant à vrai les variables des cases occupées. Chaque clause positive de région est satisfaite, puisque la région possède un chat. Chaque clause négative est satisfaite, puisque les deux cases qu'elle concerne ne sont jamais occupées simultanément. La formule est donc satisfiable si et seulement si la grille possède une solution.

Dans le cas $N = M = K = n$, les contraintes de régions imposent exactement n chats. Comme il y a n lignes et au plus un chat par ligne, chaque ligne contient nécessairement un chat. Le même raisonnement s'applique aux colonnes. Il n'est donc pas nécessaire d'ajouter des clauses positives pour chaque ligne ou colonne dans ce cas particulier.

3 Lecture de la grille et production du fichier DIMACS

3.1 Format et vérifications

La fonction `read_meowdoku` lit le format prescrit par le sujet. Elle ignore les lignes vides et les commentaires commençant par `c`. La première ligne utile indique les dimensions avec `GRID N M`, la deuxième déclare les régions avec `ZONES`, puis chaque ligne décrit les régions auxquelles appartiennent les cases. Par exemple :

```
c Exemple de format
GRID 2 3
ZONES A B
A A B
A B B
```

Cet exemple illustre le format d'entrée, sans garantir que cette grille particulière possède une solution.

Le lecteur contrôle que les dimensions sont des entiers strictement positifs, que les noms de régions sont distincts, et que la matrice possède le nombre annoncé de lignes et de colonnes. Il vérifie également que toutes les régions déclarées apparaissent et qu'aucune région non déclarée n'est utilisée. Une entrée mal formée provoque une exception `ValueError`. Une grille bien formée peut néanmoins être insatisfiable : le lecteur ne cherche pas de solution.

Le résultat de la lecture est une liste de listes de noms de régions. Cette représentation permet d'accéder directement à la région d'une case et de regrouper les cases par région dans un dictionnaire.

3.2 Numérotation des variables

DIMACS identifie les variables par des entiers strictement positifs. Nous utilisons une numérotation par lignes :

$$\mathrm{id}(i, j) = (i - 1)M + j.$$

Pour une grille de deux lignes et trois colonnes, cela donne :

$$\begin{array}{ccc} 1 & 2 & 3 \\ 4 & 5 & 6 \end{array}$$

Les indices Python commencent à zéro ; le programme utilise donc l'expression équivalente $iM + j + 1$ avec ses indices internes.

3.3 Regroupement des paires interdites

Une paire de numéros (a, b) représente deux cases qui ne peuvent pas contenir simultanément un chat. Elle devient la clause $\neg x_a \vee \neg x_b$. Toutes les paires sont conservées dans un ensemble Python, ce qui élimine les doublons. En effet, deux cases peuvent appartenir à la même ligne et à la même région : les deux règles donnent alors exactement la même clause.

Les numéros de chaque paire sont conservés dans l'ordre croissant. Les paires sont triées avant l'écriture pour produire un ordre stable des clauses. Les clauses positives de régions sont ajoutées séparément.

3.4 Écriture DIMACS

La fonction `meowdoku_to_dimacs` écrit un en-tête `p cnf V C`, où $V = NM$ et C est le nombre réel de clauses après suppression des doublons. Chaque clause est écrite sur une ligne terminée par `0`. Un entier positif représente une variable ; un entier négatif représente sa négation.

Par exemple, la paire interdite $(1, 2)$ donne :

```
-1 -2 0
```

Une région contenant les cases 1, 2 et 4 fournit notamment la clause positive :

```
1 2 4 0
```

L'exécution de la conversion peut se faire ainsi :

```
uv run python grid_convertion.py grille.txt grille.cnf
```

4 Génération d'une grille possédant une solution

4.1 Construire la solution avant les régions

Une possibilité serait de produire des régions au hasard, puis d'appeler un solveur et de recommencer lorsque la grille est insatisfiable. Nous choisissons de construire d'abord un placement valide de chats, puis de répartir les régions autour de ce placement. Cela garantit l'existence d'une solution sans faire intervenir le solveur SAT pendant la génération.

La fonction `generate_meowdoku` prend une taille n, un chemin de sortie facultatif et une graine aléatoire facultative. Elle retourne une matrice de noms de régions. Le placement témoin est utilisé en interne mais n'est pas retourné et n'est pas écrit dans le fichier de la grille.

4.2 Placement des chats par retour arrière

Nous plaçons un chat sur chaque ligne, en choisissant une colonne encore libre. Une colonne utilisée est retirée des choix possibles. Cette construction impose automatiquement un seul chat par ligne et par colonne. Les colonnes choisies forment donc une permutation.

Pour une nouvelle ligne, nous interdisons les colonnes dont la distance à la colonne du chat de la ligne précédente est égale à un. La colonne identique est déjà exclue puisqu'elle est occupée. Il reste ainsi la condition

$$|c_{i+1} - c_i| > 1,$$

où c_i désigne la colonne du chat de la ligne i. Vérifier seulement la ligne précédente suffit, car les chats des lignes plus anciennes sont éloignés d'au moins deux lignes.

Les colonnes libres sont essayées dans un ordre aléatoire. Si un choix ne permet pas de terminer le placement, il est annulé et un autre est essayé. Ce retour arrière évite d'abandonner toute la construction dès qu'une décision locale conduit à une impasse.

Pour $n = 4$, la permutation de colonnes $(2, 4, 1, 3)$, avec des coordonnées commençant à un, fournit par exemple :

$$\begin{array}{cccc} \cdot & * & \cdot & \cdot \\ \cdot & \cdot & \cdot & * \\ * & \cdot & \cdot & \cdot \\ \cdot & \cdot & * & \cdot \end{array}$$

Les différences entre colonnes consécutives sont 2, 3 et 2 : aucun chat n'est adjacent à un autre.

La taille 1 est acceptée. Les tailles 2 et 3 sont refusées : pour 2, les deux colonnes diffèrent nécessairement de un ; pour 3, la colonne centrale doit voisiner dans la permutation avec une colonne dont elle diffère de un. Pour tout $n \geq 4$, une solution existe : énumérer les colonnes paires en ordre croissant, puis les colonnes impaires en ordre croissant fournit une permutation valide. Le programme utilise néanmoins le retour arrière aléatoire afin de varier les placements.

4.3 Initialisation des régions et des candidats

Chaque case contenant un chat reçoit un nom de région distinct. Toutes les autres cases sont initialement sans région. Les noms sont les lettres de A à Z pour $n \leq 26$, puis R1, R2, etc. au-delà.

Nous enregistrons ensuite les voisins libres de chaque chat comme *candidats à la coloration*. À ce stade, ces voisins ne sont pas encore coloriés. Une candidature contient une case et la région qui pourrait l'occuper. Seuls les voisins horizontaux et verticaux sont considérés pour assurer des régions connexes par les côtés.

4.4 Croissance progressive des régions

À chaque étape, une candidature est choisie au hasard. Si sa case est encore vide, elle reçoit la région proposée. Nous ajoutons alors ses propres voisins libres aux candidats. La croissance

se propage donc au-delà des voisins immédiats des chats : chaque nouvelle case coloriée permet d'atteindre de nouvelles cases.

Une même case peut être candidate pour plusieurs régions. La première candidature effectivement traitée fixe sa région. Les autres candidatures pour cette case sont ensuite ignorées, car elle n'est plus vide. Aucune case déjà attribuée n'est recoloriée.

Le tirage porte sur les candidatures, et non directement sur les cases vides : une case proposée plusieurs fois apparaît plusieurs fois dans la liste. Cette méthode introduit de la variété, mais ne constitue pas un tirage uniforme parmi toutes les partitions possibles de la grille.